Hydro Sense: An ESP32 Edge Node with Serverless AWS Telemetry for Hydroponic Reservoir Monitoring and Threshold-Triggered Pump Control

TV Dhanush
Reg. No. 24BEL0025
Dept. of Electrical and Computer Science Engineering
School of Electrical Engineering (SEEE)
Vellore Institute of Technology (VIT)
Vellore, India

Ephraim Chacko John
Reg. No. 24BEL0068
Dept. of Electrical and Computer Science Engineering
School of Electrical Engineering (SEEE)
Vellore Institute of Technology (VIT)
Vellore, India

***Abstract*—Hydroponic reservoirs require repeated checks of solution level and dissolved-solids concentration, and a stand-alone controller provides little remote visibility or history. This paper presents Hydro Sense, a prototype that couples an ESP32 edge node with a serverless Amazon Web Services (AWS) back end. The node reads air temperature, relative humidity and pressure from a BME280 sensor, a resistive water-level signal mapped to a nominal 0–10 cm scale, a light-dependent-resistor percentage, and an analog total-dissolved-solids (TDS) estimate obtained by 30-sample median filtering, temperature compensation and the sensor vendor's cubic voltage-to-ppm conversion. Refilling is controlled locally by a threshold-triggered state machine that energizes the pump for 4 s and then waits 2 s before re-evaluating the level. Telemetry is posted through Amazon API Gateway to an AWS Lambda function that stores records in Amazon DynamoDB and publishes an Amazon SNS notification when the integer-converted water level is at most 1; a Streamlit dashboard retrieves the 50 most recent records for indicator and trend display. Photographs, dashboard captures and AWS console records confirm that the pipeline is deployed and that data reach the dashboard, but calibrated accuracy, latency and reliability have not yet been measured. A code-level audit shows that dashboard pump-control requests are not executed by the back end; remote actuation, calibrated volumetric dosing and a quantitative validation protocol are therefore defined as future work.**

***Index Terms*—Hydroponics, Internet of Things, edge computing, serverless computing, ESP32, total dissolved solids, telemetry, automated monitoring**

I. Introduction

Hydroponic production supplies mineral nutrients through an aqueous solution instead of soil, so the volume and ionic concentration of that solution must be kept within crop-dependent ranges throughout the cycle [1]. Electrical conductivity (EC) or its derived TDS value is the usual proxy for concentration, and the solution level determines whether roots and pumps remain supplied. In small installations both are often checked by hand. Surveys of agricultural Internet of Things (IoT) systems identify low-cost sensing, wireless connectivity and cloud data management as the enabling layers for automating such tasks, while also noting persistent challenges in sensor reliability, power, interoperability and data quality [2], [3].

Four engineering concerns motivated this work. First, manual monitoring requires repeated measurement and intervention. Second, when a controller acts faster than the measured quantity responds—for example, before added solution has mixed or a level reading has settled—it can overshoot its target. Third, analog probes immersed in a conductive solution are exposed to electrical noise and calibration error, and conductivity varies strongly with temperature [4]; a threshold rule driven by an uncalibrated or noisy signal may therefore trigger incorrectly. Fourth, a stand-alone controller offers no remote visibility, history or alerting. These are design motivations for the present prototype, not claims about hydroponic systems in general.

Hydro Sense addresses part of this problem space with an ESP32 microcontroller that performs sensing, filtering and refill control locally, and a serverless AWS pipeline that stores telemetry, raises low-level notifications and serves a monitoring dashboard. Published IoT hydroponic prototypes typically combine sensors, a controller, an IoT or cloud platform and pump actuation [5]–[9]. Within the studies reviewed in Section II, the behavior of the cloud request-handling tier and the boundary between demonstrated and intended capabilities are rarely documented at code level. This paper therefore reports the system as an implementation case study and grades each capability by the evidence available.

The contributions are: (i) an edge–cloud reference implementation in which refill control runs on the device independently of the network, while API Gateway, Lambda, DynamoDB and SNS provide telemetry storage, retrieval and alerting; (ii) a pulse-and-settle refill state machine that separates pump activation from re-measurement; (iii) an evidence-graded audit of the implementation, including a request-routing defect that causes dashboard pump commands to be stored as telemetry; and (iv) a validation protocol and a safety-oriented architecture for remote actuation, both identified as future work.

Section II reviews related work. Sections III–V describe the architecture, embedded implementation and cloud integration. Section VI states the evaluation status and proposed protocol, Section VII discusses observed results, Section VIII lists

limitations, Section IX outlines future work and Section X concludes.

II. Related Work

A. IoT-Based Hydroponic Monitoring and Nutrient Control

Sambo *et al.* review nutrient-solution management for soilless systems and discuss how sensors and decision support can make solution control more precise [1]. Before low-cost IoT platforms became common, Domingues *et al.* developed an automated system that controlled pH and nutrient-solution concentration and evaluated it in hydroponic lettuce production [5]. Mehra *et al.* combined IoT sensing with a deep neural network that selects control actions for a hydroponic system [6]. Lakshmanan *et al.* used a NodeMCU with Node-RED and MQTT to send hydroponic data to a cloud web page and mobile application, and tested the prototype in two environments [7]. Closer to the present hardware, Abu Sneineh and Shabaneh built an ESP32 system with TDS, pH, water-level and temperature sensing in which pumps can be run manually from a Blynk smartphone application or automatically by the ESP32 [8]. Ariyanto *et al.* used an ESP32 with pH, TDS, temperature and level sensors and a hysteresis rule that switches a nutrient pump on below 1000 ppm and off at 1400 ppm; they reported average TDS and pH sensor errors below 0.4% and a five-day trial [9].

B. TDS Measurement and Sensor Calibration

TDS probes measure conductivity, which increases by roughly 2% per °C near room temperature, so readings are normally referred to 25 °C [4]. The vendor reference procedure for the analog TDS module used here averages repeated samples with a median filter, divides the measured voltage by a linear temperature coefficient and converts it with a cubic polynomial; the vendor specifies a 0–1000 ppm range and ±10% full-scale accuracy at 25 °C, and notes that the reference code assumes 25 °C when no temperature sensor is fitted [10]. Median filtering is used because it suppresses impulsive outliers while preserving step changes [11]. The ESP32 analog-to-digital converter (ADC) adds its own uncertainty: Espressif documents a reference-voltage spread of 1000–1200 mV between devices and attenuation-dependent ranges of best accuracy, which motivates per-device calibration [12].

C. Edge Processing and Serverless Back Ends

Edge computing places computation near the data source to reduce latency and bandwidth and to tolerate degraded connectivity [13]. Serverless function-as-a-service platforms run event-driven functions without server management and bill per invocation, but bring constraints such as statelessness, cold-start delay and limited execution control [14], [15]. Glikson *et al.* proposed extending the serverless model to edge devices [16]. Hydro Sense uses the simpler split: deterministic control on the microcontroller and stateless cloud functions for data handling.

D. Research Gap

Table I summarizes the comparison. The reviewed systems [6]–[9] demonstrate integrated sensing and actuation on IoT platforms, and some report sensor validation [9]. Hydro Sense does not exceed them in validation depth. Its intended contribution is narrower: a fully serverless AWS telemetry path for a hydroponic edge node, documented together with an explicit separation of verified, partially evidenced and unimplemented functions. No novelty is claimed for the use of standard sensors, an ESP32, AWS services or Streamlit as such.

TABLE I
Comparison with Related Hydroponic IoT Systems

Work	Controller / platform	Actuation	Reported validation
[5]	Automated pH/EC control system	pH and nutrient dosing	Hydroponic lettuce production
[6]	IoT sensing with DNN decision model	DNN-selected control actions	As reported in [6]
[7]	NodeMCU; Node-RED, MQTT; web and mobile app	Bot for supply-chain control and notifications	Prototype tests in two environments
[8]	ESP32; Blynk app	Manual (app) and automatic pumps	Experimental data reported
[9]	ESP32; web dashboard	TDS hysteresis pump (1000/1400 ppm)	Sensor error vs. reference; 5-day trial
This work	ESP32; API Gateway, Lambda, DynamoDB, SNS; Streamlit	Local timed refill only; no remote actuation	Functional evidence only (Section VI)

III. System Architecture

A. Overview

Fig. 1 shows the implemented architecture. It has three planes. The *local plane* acquires and processes sensor data, drives the display and runs refill control on the ESP32 without any dependence on the network. The *cloud plane* receives telemetry through API Gateway, stores it in DynamoDB and raises SNS notifications from a single Lambda function. The *monitoring plane* is a Streamlit dashboard that reads telemetry through the same endpoint. No path from the dashboard to the pump is drawn because none is implemented (Section V-F).

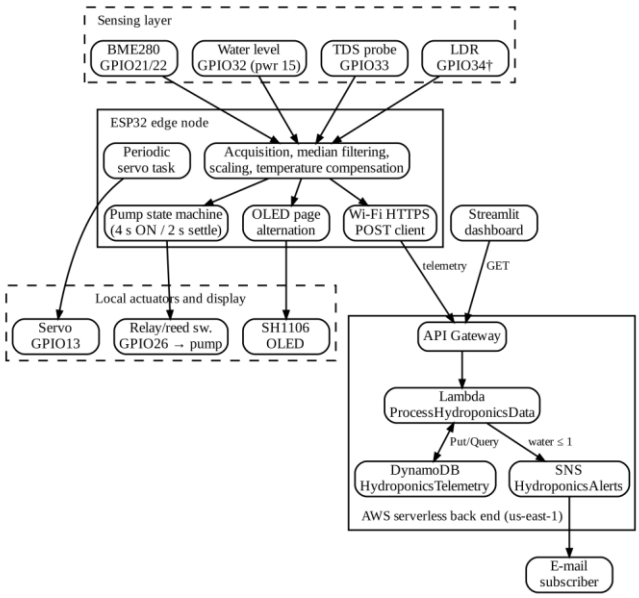


Fig. 1. Implemented Hydro Sense architecture. Solid groups are implemented functions; dashed groups are sensor and actuator hardware. The dashboard has read-only access. †The LDR pin differs between the firmware (GPIO34) and the presentation (GPIO27); see Section III-B.

B. Hardware

Table II lists the hardware and pin assignments, cross-referenced between the project presentation and the firmware. The ESP32 reads a BME280 environmental sensor over I²C (SDA GPIO21, SCL GPIO22), a resistive REL-35 water-level sensor whose supply is switched from GPIO15 and whose output is read on GPIO32, an analog TDS module on GPIO33 and a light-dependent-resistor (LDR) module. GPIO26 drives the pump-switching element, which is named a reed switch in the firmware and listed as a one-channel relay module in the component list; GPIO13 drives a servo, and an SH1106 OLED displays readings. The prototype (Fig. 2a) uses two plastic vessels connected by flexible tubing, with the level sensor mounted in the upper vessel, a four-cell AA battery holder and breadboard-mounted modules.

One pin assignment is inconsistent. The firmware reads the LDR (`photoin`) on GPIO34, whereas the presentation lists GPIO27. The two are not interchangeable on a Wi-Fi-connected ESP32: GPIO27 is ADC2 channel 7, and ADC2 cannot be read while the Wi-Fi driver is running, whereas GPIO32–39 belong to ADC1 [12]. GPIO34 is therefore the assignment consistent with the networked firmware, but the physical wiring must be confirmed before the light data are interpreted.

TABLE II
HARDWARE COMPONENTS AND PIN MAPPING

Component	Connection	Source / status
ESP32 (Wi-Fi MCU)	—	Presentation; firmware
BME280 (T, RH, p)	I²C: SDA GPIO21, SCL GPIO22	Presentation; firmware
REL-35 resistive water sensor	Signal GPIO32; supply GPIO15	Presentation; firmware
Analog TDS module	GPIO33	Firmware (listed, no pin, in presentation)
LDR module	GPIO34 (firmware) / GPIO27 (presentation)	Conflict; wiring to be confirmed
Pump switch (relay / reed switch)	GPIO26	Presentation; firmware
Servo	GPIO13	Firmware
SH1106 OLED	I²C	Presentation; photographs
Pump, battery pack, power converter, tubing	—	Circuit sketch; photographs

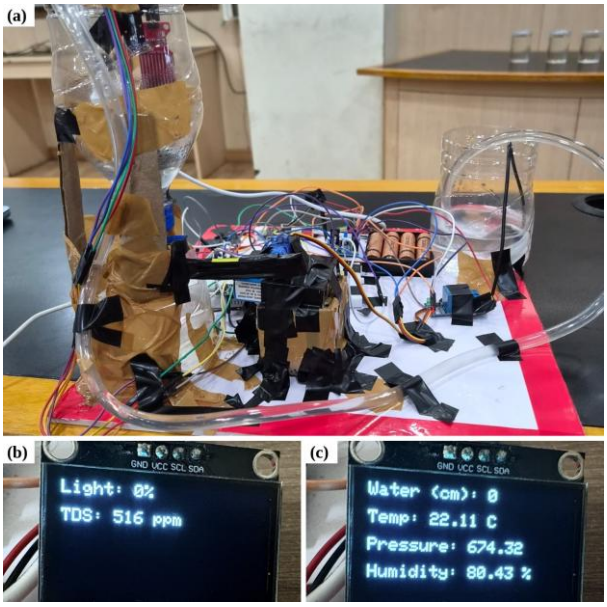


Fig. 2. Hydro Sense prototype. (a) Bench assembly with the level sensor in the upper vessel, tubing, switching modules and battery pack. (b), (c) The two alternating OLED pages, showing light and TDS, then water level, temperature, pressure and humidity. Displayed values are uncalibrated (Section VII-A).

Fig. 3 reproduces the original hand-drawn block diagram from the project presentation. It is a conceptual sketch rather than a wiring schematic: it shows the ESP32, water sensor, BME280, photoresistor, power converter, battery, reed switch and pump, and three additional motors (illumination adjustment, solution drain and nutrient feed). It does not show the TDS module or the OLED, and the supplied firmware drives only one pump-switch output and one servo. Only the elements confirmed by the firmware are described as implemented in this paper.

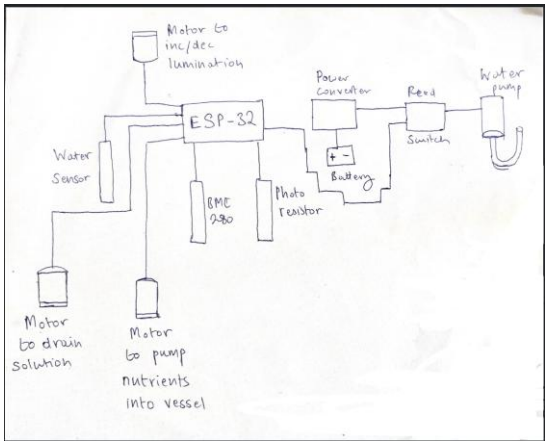


Fig. 3. Original conceptual circuit diagram from the project presentation. The drain and nutrient motors shown are not driven by the supplied firmware.

C. Firmware Organization

The firmware is an Arduino-framework program built around a single main loop (Fig. 4). Each pass acquires all sensors, converts and filters the readings, updates the OLED, advances the pump state machine and the periodic servo task, and, when an upload is due and Wi-Fi is connected, posts a JSON telemetry record over HTTPS. Because refill control runs on the device, it continues when the network is unavailable.

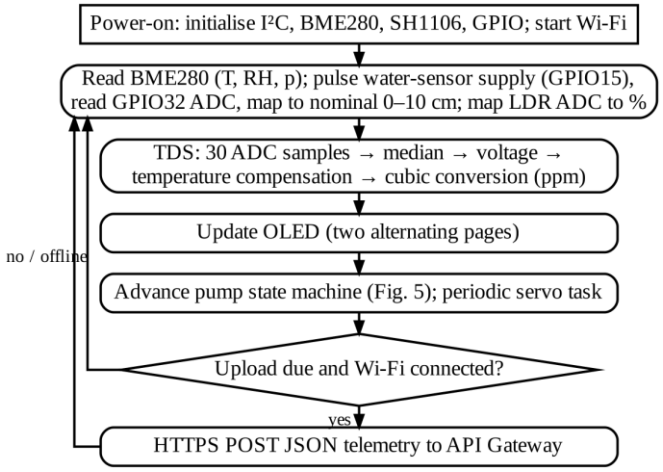


Fig. 4. Simplified logical processing sequence of the ESP32 firmware. The exact ordering and timing of stages within the loop follow the firmware source.

IV. Methodology and Implementation

A. Environmental Sensing

Temperature, relative humidity and pressure are read from the BME280 through its driver library; pressure is divided by 100 to express it in hectopascals. The manufacturer specifies ±1.0 °C, ±3% RH (20–80% RH at 25 °C) and ±1.0 hPa absolute accuracy [17]; these figures were not verified in the prototype. Pressure is displayed locally but is not part of the uploaded telemetry record.

B. Water-Level Estimation

The resistive level sensor is energized from GPIO15 only during a reading; in the presentation firmware the supply is raised, the ADC is sampled after 10 ms and the supply is removed again, which limits continuous current through the immersed traces. The raw count N_w from GPIO32 is mapped linearly to a nominal 0–10 cm scale,

$$h = 10\,(N_w - N_0) / (N_1 - N_0)\ \text{cm}, \tag{1}$$

where N_0 and N_1 are constants in the firmware. The response of an exposed-trace resistive sensor depends on immersion depth and on solution conductivity, and no calibration against a reference scale has been recorded. The value h is therefore a nominal index and not a calibrated physical length.

C. Light Level

The LDR count N_L (0–4095 for the 12-bit ADC) is scaled to an integer percentage,

$$L = \lfloor 100\,N_L / 4095 \rfloor\ \%, \tag{2}$$

using the Arduino `map` function, which truncates. L is a relative indicator; its direction (whether brighter light raises or lowers N_L) depends on the divider in the LDR module and has not been characterized.

D. TDS Measurement

Thirty ADC samples are taken from GPIO33 and their median N_{med} is used, which rejects isolated spikes [11]. The median is converted to a voltage and corrected with the linear temperature coefficient of the vendor reference procedure [10], [4]:

$$V = N_{med}\,V_{ref} / N_{FS},\quad k = 1 + 0.02\,(T - 25),\quad V_c = V / k, \tag{3}$$

and the compensated voltage is converted with the vendor's cubic polynomial,

$$\text{TDS} = 0.5\,(133.42\,V_c^3 - 255.86\,V_c^2 + 857.39\,V_c)\ \text{ppm}. \tag{4}$$

Two points qualify these equations. First, the vendor code assumes a 5.0 V reference and a 10-bit full scale [10]; on the ESP32, V_{ref} and N_{FS} must reflect the 3.3 V supply, the selected attenuation and the 12-bit range, and device-specific ADC deviation remains [12]. Second, the documented hardware contains no immersed temperature probe, so T in (3) can only be the BME280 air temperature or a fixed value; either is a proxy for solution temperature. No comparison against a reference TDS or EC meter has been performed, so the accuracy of the reported ppm values is unknown.

TABLE III
Sensor Processing as Implemented

Quantity	Processing	Calibration status
T, RH, p	BME280 library; p ÷ 100 → hPa	Datasheet only [17]
Water level	Switched supply; linear map (1), nominal 0–10 cm	Uncalibrated
Light	Integer map (2), 0–100 %	Relative; polarity unverified
TDS	30-sample median; (3)–(4)	No reference comparison

E. Threshold-Triggered Refill Control

Refill control is the pulse-and-settle state machine of Fig. 5. In CHECK, the nominal level h is compared with a firmware threshold h_{th}. If $h < h_{th}$, the switching output is activated for 4 s (PUMP_ON) and then deactivated for 2 s (SETTLE) before the level is checked again; otherwise the pump stays off. Bounding

each activation and re-measuring only after a settling interval is intended to reduce overshoot caused by the delayed response of the level reading, which corresponds to the delayed-feedback concern in Section I. An earlier firmware version shown in the presentation instead held the output active for as long as the raw count was below 50, i.e., continuous pumping until the threshold was crossed.

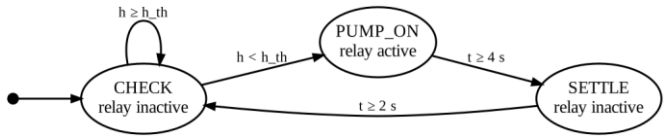


Fig. 5. Local pump state machine. h is the nominal water level from (1) and h_{th} the firmware threshold.

This controller is timed, not volumetric. The volume delivered in one pulse, $V_p = \int Q(t)\,dt$ over the 4 s interval, depends on the pump flow rate Q, the hydraulic head, tubing losses and the supply voltage, which falls as the battery discharges. None of these was measured, so the system does not deliver a known or calibrated dose; it delivers repeated bounded pump activations until the level threshold is met. The pump fills the vessel with whatever solution is in the supply reservoir; the firmware does not adjust TDS by dosing concentrate.

F. Display, Servo and Connectivity

The OLED alternates between two pages (Fig. 2b, c): light and TDS on one, and water level, temperature, pressure and humidity on the other. The firmware also moves a servo on GPIO13 at a fixed period; its mechanical role is not documented in the supplied materials, and the conceptual diagram (Fig. 3) associates a motor with illumination adjustment. At start-up the firmware attempts a Wi-Fi connection; uploads are made periodically only when a connection exists, so sensing, display and refill control continue offline. Each upload is a JSON object with the fields `device_id`, `water_cm`, `temp`, `humidity`, `light_percent` and `tds_ppm`.

V. Cloud Integration and Monitoring

A. Request Handling

An API Gateway endpoint invokes the Lambda function `ProcessHydroponicsData` (Fig. 8a). With proxy integration, API Gateway passes the HTTP method and the raw request body to the function, which must return a status code, headers and body [18]. Fig. 6 summarizes the handler. A GET request queries telemetry; every other method is treated as a telemetry submission. Table IV lists the responsibilities of each service.

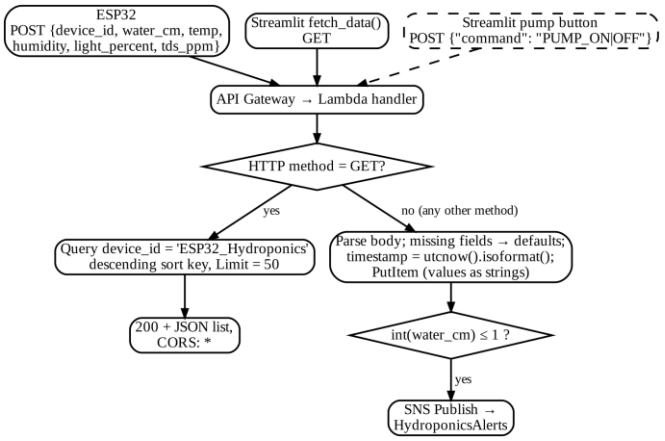


Fig. 6. Request handling in the Lambda function. The dashed path is a dashboard pump command, which is routed into the telemetry branch because the handler distinguishes requests only by HTTP method.

TABLE IV
AWS Service Responsibilities and Evidence

Service	Role in Hydro Sense	Console evidence (Fig. 8)
API Gateway	HTTPS endpoint for device POST and dashboard GET	Trigger of the Lambda function
Lambda	Parse, store, query, alert	Function exists, us-east-1, no layers
DynamoDB	Table `HydroponicsTelemetry`	Keys device_id (S), timestamp (S); on-demand; 11 items
SNS	Topic `HydroponicsAlerts`	Standard topic; one confirmed e-mail subscription

B. Data Schema and Storage

For a submission, the handler parses the body, reads the six telemetry fields (substituting default values for any that are missing), creates a server-side timestamp with `datetime.datetime.utcnow().isoformat()` and writes the item with the sensor values converted to strings. The table uses `device_id` (String) as partition key and `timestamp` (String) as sort key (Fig. 8b), which matches the query in Section V-C. ISO-8601 strings in a consistent format sort chronologically under DynamoDB's byte-order comparison [19]. The timestamps carry no time-zone designator, so clients must treat them as UTC. Because values are stored as strings, every consumer must convert them to numbers before analysis.

C. Telemetry Retrieval

A GET request queries items whose `device_id` equals `ESP32_Hydroponics`, in descending sort-key order with a limit of 50, and returns them as JSON with permissive cross-origin (CORS) headers. DynamoDB stops a query at the item limit or at 1 MB and returns `LastEvaluatedKey` for pagination [19]; the handler does not paginate, so at most the 50 newest records are visible. The device identifier is fixed in the query, which restricts the deployment to one device.

D. Low-Level Alerting

After each write, the handler publishes to the SNS topic when `int(water_cm) <= 1`. For a numeric value, `int()` truncates toward zero, so any level below 2.0 on the nominal scale triggers a notification; for a string such as "0.6" the

conversion would raise an exception. E-mail endpoints receive messages only after the recipient confirms the subscription [20]; Fig. 8c shows one confirmed subscription. The handler contains no deduplication, rate limiting or hysteresis, so a level that stays low produces one notification per upload, and the supplied code shows no explicit handling of DynamoDB or SNS errors.

E. Streamlit Dashboard

The dashboard (Fig. 7) is a Python application using Streamlit, Requests and Pandas. Its `fetch_data()` function issues a GET to the API endpoint and, if the response has status 200 and contains a JSON list, builds a DataFrame, converts the five sensor fields to numbers, parses timestamps and sorts by time. The last row supplies the indicator values. The page shows two indicator rows—water level, temperature and humidity in the first; water level, temperature, humidity, light and TDS in the second, so three quantities appear twice—and two time-indexed line charts: temperature with humidity, and water level with TDS. Light level has no trend chart. A sidebar slider (5–60 s, default 10 s) and a checkbox (enabled by default) control auto-refresh, implemented by sleeping for the selected interval and calling `st.rerun()`, which restarts the script [21]. The slider sets the dashboard polling period; it is not a measurement of end-to-end latency. Table V summarizes these features.



Fig. 7. Streamlit dashboard as captured during operation, showing both indicator rows and the two trend charts. The values are discussed in Section VII-A.

TABLE V
DASHBOARD CAPABILITIES

Feature	Implementation	Limitation
Data source	GET via API Gateway; 50 newest records	No pagination; single device
Indicators	Two rows from the latest record	Three metrics duplicated
Trends	T + RH; water level + TDS	No light chart
Refresh	sleep + `st.rerun()`, 5–60 s	Polling period, not latency
Pump buttons	POST {"command": …}	Not executed (Section V-F)

F. Dashboard Pump Requests: Not Implemented as Control

The dashboard sidebar contains *Turn Pump ON* and *Turn Pump OFF* buttons that POST `{"command": "PUMP_ON"}` or `{"command": "PUMP_OFF"}` to the same endpoint and display a success message whenever the request does not raise an exception; the HTTP status and response body are not checked. The Lambda handler has no command branch, no control table, no mechanism through which the ESP32 retrieves commands and no actuator acknowledgement. A command POST therefore follows the telemetry path of Fig. 6: default values replace the absent sensor fields, a record that does not correspond to any measurement is written to `HydroponicsTelemetry`, and if the default water level is at most 1 a low-level notification is also published. Remote pump control is consequently not a capability of the current system. The dashboard capture in Fig. 7 does not show the buttons, which suggests that it predates their addition.

VI. EXPERIMENTAL SETUP AND EVALUATION STATUS

A. Available Evidence

The evidence available for this paper consists of prototype photographs (Fig. 2), two OLED photographs, one dashboard capture (Fig. 7), AWS console captures of the Lambda function, DynamoDB table and SNS topic (Fig. 8), the project presentation and the firmware, dashboard and Lambda source descriptions. Account identifiers and the subscriber e-mail address are redacted in Fig. 8. No serial logs, calibration records, reference-instrument comparisons, CloudWatch logs, timing measurements or power measurements were available. Table VI lists what each item shows and what it does not.

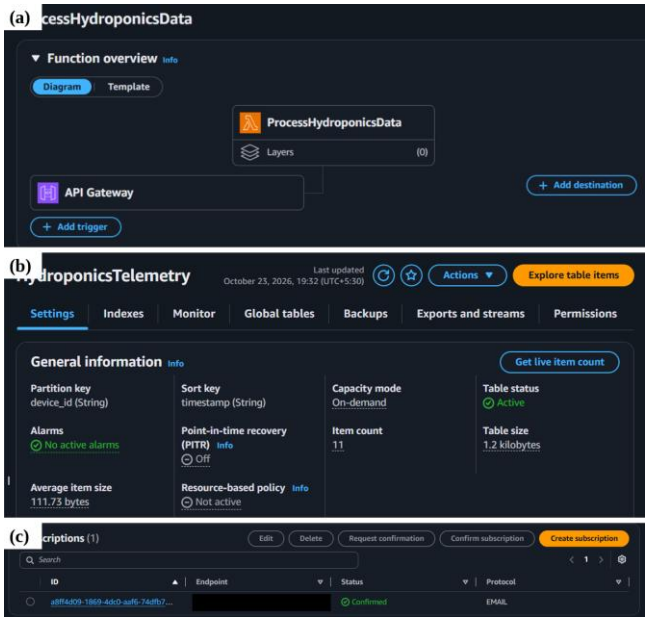


Fig. 8. AWS console evidence (identifiers redacted). (a) Lambda function with API Gateway trigger. (b) DynamoDB table key schema and status (captured 23 Oct. 2026, 19:32 IST). (c) SNS topic with one confirmed e-mail subscription.

TABLE VI
FUNCTIONAL OBSERVATIONS FROM AVAILABLE EVIDENCE

Evidence	Shows	Does not show
Prototype photos	Physical integration of sensors, switching modules, pump and vessels	Pump timing, delivered volume
OLED photos	Local acquisition and both display pages	Accuracy of any value
Dashboard capture	Records retrieved through the API and plotted	Delivery rate; latency
Lambda / API Gateway	Function deployed with HTTP trigger	Invocation or error counts

Evidence	Shows	Does not show
DynamoDB	Compatible key schema; 11 stored items	Record content or rate
SNS	Confirmed e-mail subscription	That an alert was sent or received

B. Proposed Evaluation Protocol

Because quantitative evidence is not yet available, Table VII defines the tests required to validate the implementation. All entries are proposed and have not been performed. Each sensor test should report mean error, standard deviation and maximum error over repeated trials at several operating points; each network test should report distributions rather than single values.

TABLE VII
PROPOSED EVALUATION (NOT YET PERFORMED)

Test	Method and reference	Metric
Water-level calibration	Graduated scale at ≥5 depths; fit (1)	Error (cm); hysteresis
TDS accuracy	Reference EC/TDS meter; standard solutions; 15–35 °C	Error (ppm, %FS)
T / RH accuracy	Reference hygrometer and thermometer	Error (°C, %RH)
Pressure check	Local station or reference barometer	Error (hPa)
Pump repeatability	Weigh output of $n \geq 30$ pulses at several battery voltages	Mean, SD of volume per pulse
Refill stability	Log h during refill	Overshoot; pulses to threshold
Telemetry delivery	Sequence-numbered uploads vs. stored items	Success rate
API response time	Timed GET/POST; CloudWatch duration	Median, 95th percentile
Alert delivery	Forced low-level record to e-mail receipt	Delay; duplicate count
Network failure	Disable Wi-Fi during refill	Local control continuity
Power	Inline current measurement per state	Average and peak current

VII. RESULTS AND DISCUSSION

A. Observed Functional Results

The results are qualitative. The OLED photographs (Fig. 2b, c) show light 0%, TDS 516 ppm, water level 0 cm, temperature 22.11 °C, pressure 674.32 and humidity 80.43%. The temperature and humidity are plausible indoor values. The pressure is not: for a site near Vellore, a reading close to 1000 hPa would be expected, so 674.32 indicates an unresolved sensor, configuration or scaling issue. The TDS value lies inside the module's 0–1000 ppm range [10] but has no reference comparison.

The dashboard capture (Fig. 7) shows water level 0 cm, temperature 24.1 °C, humidity 100.0%, light 100.0% and TDS 0.0 ppm, with flat trends over the displayed window. Several of these values sit at range limits. Humidity of 100% may reflect condensation or saturation; TDS of 0 may indicate a dry probe, a disconnected input or substituted default values; light of 100% may indicate ADC saturation or inverted polarity. These causes cannot be distinguished with the available evidence. The water level of 0 cm on both the OLED and the dashboard satisfies the alert condition, so each such record would also produce an SNS notification.

These observations establish end-to-end data flow from the device, through API Gateway, Lambda and DynamoDB, to the dashboard, and the deployed configuration shown in Fig. 8. They do not establish the accuracy of any measurement, the rate or latency of delivery, the timing of the pump state machine or the receipt of alerts.

B. Discussion

Edge versus cloud responsibilities. Keeping refill control on the device means that a network or cloud outage does not stop refilling, consistent with the rationale for edge processing [13]. The cost is that the control parameters (h_{th}, 4 s, 2 s) are compiled into the firmware and cannot be changed or audited remotely.

Serverless back end. A single Lambda function with an on-demand table avoids server administration and suits the low, bursty request rate of one device [14], [15]. The same design concentrates read, write and alert logic in one handler that distinguishes requests only by HTTP method. The command-routing defect in Section V-F follows directly from that choice: any new request type is silently treated as telemetry. Separate routes with schema validation would remove this failure mode.

Data handling. Storing values as strings and taking only the 50 newest items keep the implementation simple but shift numeric conversion and history management to the client; long-term analysis would require pagination or export. Polling with `st.rerun()` is simple, but each refresh re-queries the table regardless of whether new data exist.

Control design. The pulse-and-settle rule bounds each activation, but without hysteresis a level hovering near h_{th} can still cause frequent short pump cycles. The hysteresis band used in [9] for TDS control is a natural comparison and a candidate improvement.

VIII. LIMITATIONS AND THREATS TO VALIDITY

Calibration and conversion. The water-level map (1), the light map (2) and the TDS conversion (3)–(4) have not been calibrated against references. The ESP32 ADC has device-dependent reference voltage and nonlinear regions [12], and the vendor TDS procedure was written for a different ADC and reference [10].

Noise and temperature. Median filtering removes isolated spikes but not drift, polarization or ground-loop effects in conductive solutions. TDS compensation uses a proxy temperature, and conductivity changes by about 2% per °C [4].

Timed pumping. A fixed run time does not define a delivered volume; flow depends on head and battery voltage, so the term *volumetric dosing* is not applicable to the present implementation.

Network dependence. Telemetry, alerts and the dashboard require connectivity; records produced while offline are not described as buffered.

Missing cloud-to-device control. No command path to the ESP32 exists. Dashboard command requests create misleading telemetry records and may trigger false alerts.

API validation and security. Wildcard CORS headers permit cross-origin browser access but provide no authentication or authorization. The supplied code shows no input-schema validation, no type checking before `int()` and no explicit exception handling for DynamoDB or SNS failures.

Alert reliability. Truncation shifts the effective threshold to below 2.0, and the absence of deduplication or hysteresis can produce repeated notifications.

Evidence. No quantitative measurements of accuracy, latency, delivery rate, alert delay, pump repeatability or power were available, and the dashboard capture may predate the current dashboard code. The presentation firmware differs from the firmware described here, and the LDR pin conflict is unresolved.

IX. Future Work

Every item in this section is proposed work that has not been implemented or tested.

A. Remote Pump Control

Remote pump actuation is **not implemented in the supplied code**. Future work will implement secure bidirectional communication between the dashboard and the ESP32 (Fig. 9). The proposed design adds an explicit, authenticated command endpoint with schema validation; cloud storage of the desired pump state with a command identifier and expiry time; periodic retrieval of the desired state by the ESP32; local safety checks before execution, including level limits, a maximum run time and sensor-plausibility tests; command acknowledgement; and reporting of the actual actuator state so that the dashboard can display desired and reported states separately. A managed device shadow, whose documents hold desired, reported and delta state [22], is one candidate for the state store. The function will be validated experimentally in a later development phase.

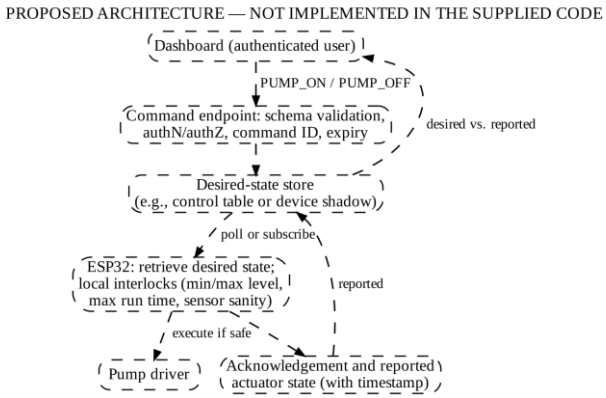


Fig. 9. Proposed remote-control architecture (future work, not implemented). The ESP32 retains authority to refuse unsafe commands.

B. Closed-Loop Volumetric Dosing

Calibrated dosing requires a measured delivered volume, for example from a flow sensor or a characterized peristaltic pump, combined with feedback from the TDS and level readings and explicit mixing and settling intervals, so that dosing repeatability can be quantified with the protocol in Table VII.

C. Crop-Stage-Aware Nutrient Management

A pH sensor and stored crop profiles could allow pH and TDS setpoints to change with crop type and growth stage instead of fixed thresholds.

D. Plant Health Monitoring

An ESP32-CAM module could capture periodic plant images for image-analysis methods aimed at early detection of nutrient-deficiency symptoms.

E. Reliability and Security

Planned improvements include authenticated API access, TLS certificate verification on the device, input-schema validation, numeric storage types, pagination, alert deduplication and hysteresis, command time-outs, offline buffering and explicit fault recovery in the Lambda handler.

X. Conclusion

Hydro Sense integrates an ESP32 edge node with a serverless AWS pipeline for hydroponic reservoir monitoring. The node performs median-filtered, temperature-compensated TDS estimation, nominal level and light sensing, environmental sensing, local display and a pulse-and-settle refill controller that runs independently of the network. The cloud tier stores telemetry in DynamoDB, serves the most recent 50 records to a Streamlit dashboard and publishes SNS notifications for low nominal levels. Deployment evidence confirms the data path from device to dashboard. The study also documents what the system does not yet do: its measurements are uncalibrated, its pump control is timed rather than volumetric, and dashboard pump requests are not executed and instead create misleading records. Remote pump control remains a future extension, to be built on authenticated commands, device-side safety checks and acknowledged actuator state, and evaluated with the quantitative protocol defined in this paper.

Author Contributions

TV Dhanush developed the AWS cloud pipeline, including API Gateway integration, Lambda processing, DynamoDB telemetry storage and SNS notification, and the Streamlit dashboard and cloud-side integration. Ephraim Chacko John designed the hardware circuit and performed sensor integration, ESP32 firmware development, OLED display, motor and actuator interfacing, prototype construction and calibration activities.

References

[1] P. Sambo *et al.*, “Hydroponic solutions for soilless production systems: Issues and opportunities in a smart agriculture perspective,” *Front. Plant Sci.*, vol. 10, Art. no. 923, 2019, doi: 10.3389/fpls.2019.00923.

[2] A. Tzounis, N. Katsoulas, T. Bartzanas, and C. Kittas, “Internet of Things in agriculture, recent advances and future challenges,” *Biosyst. Eng.*, vol. 164, pp. 31–48, 2017, doi: 10.1016/j.biosystemseng.2017.09.007.

[3] O. Elijah, T. A. Rahman, I. Orikumhi, C. Y. Leow, and M. N. Hindia, “An overview of Internet of Things (IoT) and data analytics in

agriculture: Benefits and challenges," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3758–3773, Oct. 2018, doi: 10.1109/JIOT.2018.2844296.

[4] M. Hayashi, "Temperature-electrical conductivity relation of water for environmental monitoring and geophysical data inversion," *Environ. Monit. Assess.*, vol. 96, pp. 119–128, 2004, doi: 10.1023/B:EMAS.0000031719.83065.68.

[5] D. S. Domingues, H. W. Takahashi, C. A. P. Camara, and S. L. Nixdorf, "Automated system developed to control pH and concentration of nutrient solution evaluated in hydroponic lettuce production," *Comput. Electron. Agric.*, vol. 84, pp. 53–61, 2012, doi: 10.1016/j.compag.2012.02.006.

[6] M. Mehra, S. Saxena, S. Sankaranarayanan, R. J. Tom, and M. Veeramanikandan, "IoT based hydroponics system using deep neural networks," *Comput. Electron. Agric.*, vol. 155, pp. 473–486, 2018, doi: 10.1016/j.compag.2018.10.015.

[7] R. Lakshmanan, M. Djama, S. Perumal, and R. Abdulla, "Automated smart hydroponics system using internet of things," *Int. J. Elect. Comput. Eng.*, vol. 10, no. 6, pp. 6389–6398, Dec. 2020, doi: 10.11591/ijece.v10i6.pp6389-6398.

[8] A. Abu Sneineh and A. A. A. Shabaneh, "Design of a smart hydroponics monitoring system using an ESP32 microcontroller and the Internet of Things," *MethodsX*, vol. 11, Art. no. 102401, Dec. 2023, doi: 10.1016/j.mex.2023.102401.

[9] D. Ariyanto *et al.*, "Implementation and evaluation of an IoT-based nutrient monitoring and pump control system for Pakcoy hydroponics," *J. Comput. Electron. Telecommun.*, vol. 7, no. 1, Jul. 2026, doi: 10.52435/complete.v7i1.11305.

[10] DFRobot, "Gravity: Analog TDS sensor/meter for Arduino (SKU: SEN0244)," product wiki and example code. Accessed: Oct. 23, 2026. [Online]. Available: https://wiki.dfrobot.com/sen0244/

[11] N. C. Gallagher, Jr. and G. L. Wise, "A theoretical analysis of the properties of median filters," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. ASSP-29, no. 6, pp. 1136–1141, Dec. 1981, doi: 10.1109/TASSP.1981.1163708.

[12] Espressif Systems, "Analog to digital converter," *ESP-IDF Programming Guide*, v4.2 (ESP32). Accessed: Oct. 23, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v4.2/esp32/api-reference/peripherals/adc.html

[13] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016, doi: 10.1109/JIOT.2016.2579198.

[14] I. Baldini *et al.*, "Serverless computing: Current trends and open problems," in *Research Advances in Cloud Computing*, S. Chaudhary, G. Somani, and R. Buyya, Eds. Singapore: Springer, 2017, pp. 1–20, doi: 10.1007/978-981-10-5026-8_1.

[15] E. Jonas *et al.*, "Cloud programming simplified: A Berkeley view on serverless computing," Univ. California, Berkeley, Tech. Rep. UCB/EECS-2019-3, Feb. 2019.

[16] A. Glikson, S. Nastic, and S. Dustdar, "Deviceless edge computing: Extending serverless computing to the edge of the network," in *Proc. 10th ACM Int. Syst. Storage Conf. (SYSTOR)*, Haifa, Israel, 2017, doi: 10.1145/3078468.3078497.

[17] Bosch Sensortec, "BME280: Combined humidity and pressure sensor," Datasheet BST-BME280-DS002-15, rev. 1.6, Sep. 2018.

[18] Amazon Web Services, "Lambda proxy integrations in API Gateway," *Amazon API Gateway Developer Guide*. Accessed: Oct. 23, 2026. [Online]. Available: https://docs.aws.amazon.com/apigateway/latest/developerguide/set-up-lambda-proxy-integrations.html

[19] Amazon Web Services, "Query," *Amazon DynamoDB API Reference*. Accessed: Oct. 23, 2026. [Online]. Available: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Query.html

[20] Amazon Web Services, "Amazon SNS email subscription setup and management," *Amazon SNS Developer Guide*. Accessed: Oct. 23, 2026. [Online]. Available: https://docs.aws.amazon.com/sns/latest/dg/sns-email-notifications.html

[21] Streamlit, "st.rerun," *Streamlit API Reference*. Accessed: Oct. 23, 2026. [Online]. Available: https://docs.streamlit.io/develop/api-reference/execution-flow/st.rerun

[22] Amazon Web Services, "Device Shadow service documents," *AWS IoT Core Developer Guide*. Accessed: Oct. 23, 2026. [Online]. Available: https://docs.aws.amazon.com/iot/latest/developerguide/device-shadow-document.html